

Partial Delivery: Formal Design of the Learning Task

Project: Heart Disease Prediction using Multilayer Perceptron

Juan Manuel Ruiz Llamas

November 18, 2025

Abstract

This document presents the mathematical formalization and technical design of the supervised machine learning task for the binary classification of heart pathologies, utilizing a Multilayer Perceptron (MLP) implemented from scratch and validated comparatively.

1 Used ML Technique

The primary technique selected is a **Feedforward Artificial Neural Network**, specifically a **Multilayer Perceptron (MLP)**. The MLP is a universal function approximator capable of capturing complex non-linear relationships between clinical input features and the target diagnosis. The model consists of an input layer, one or more hidden layers with non-linear activation functions (ReLU), and an output layer with sigmoidal activation to produce a probability. A *from-scratch* version using vectorized linear algebra and a reference version using the `scikit-learn` library are implemented.

2 Design of the Learning Task

The task is defined as a **Supervised Binary Classification** problem.

- **Input Domain ($\mathcal{X}$):** A set of processed clinical feature vectors derived from the *UCI Heart Disease* dataset. Following pre-processing (imputation, *One-Hot Encoding*, and standardization), each instance $x^{(i)}$ is a vector in $\mathbb{R}^{20}$.
- **Output Domain ($\mathcal{Y}$):** A set of binary labels $\{0, 1\}$, where 0 indicates the absence of pathology and 1 indicates the presence of heart disease.
- **Data Source:** A labeled dataset $D = \{(x^{(1)}, y^{(1)}), \dots, (x^{(m)}, y^{(m)})\}$, stratified into training (80%) and testing (20%) sets.

3 Target Function (Formal Definition)

The ideal target function (or concept function) is the true, unknown probabilistic relationship f that maps a patient's physiological features to their actual health status:

$$f : \mathbb{R}^n \rightarrow \{0, 1\} \tag{1}$$

Where $n = 20$ is the number of input features. Our objective is to find a hypothesis h such that $h(x) \approx f(x)$ for any unobserved x , minimizing the generalization error.

4 Learning Hypothesis

The hypothesis $h_\theta(x)$ is the parametric function defined by the neural network architecture. Given that this is a binary classification problem, the hypothesis approximates the conditional probability $P(y = 1|x; \theta)$.

For our optimized base architecture [20, 16, 1] (20 inputs, 16 hidden neurons, 1 output), the hypothesis is mathematically defined by the composition of functions:

1. Hidden Layer (Linear Propagation and Activation):

$$z^{[1]} = W^{[1]}x + b^{[1]} \quad (2)$$

$$a^{[1]} = \text{ReLU}(z^{[1]}) = \max(0, z^{[1]}) \quad (3)$$

Where $W^{[1]} \in \mathbb{R}^{16 \times 20}$ and $b^{[1]} \in \mathbb{R}^{16}$.

2. Output Layer (Prediction):

$$z^{[2]} = W^{[2]}a^{[1]} + b^{[2]} \quad (4)$$

$$h_\theta(x) = a^{[2]} = \sigma(z^{[2]}) = \frac{1}{1 + e^{-z^{[2]}}} \quad (5)$$

Where $W^{[2]} \in \mathbb{R}^{1 \times 16}$, $b^{[2]} \in \mathbb{R}^1$, and $\sigma(\cdot)$ is the Sigmoid function.

5 Representation of the Learned Function

The learned function is represented by the final set of parameters (weights and biases) $\theta = \{W^{[1]}, b^{[1]}, W^{[2]}, b^{[2]}\}$ that minimize the cost function, as shown in figure number 1.

6 Learning Algorithm

The learning process seeks the optimal parameters θ by minimizing the **Binary Cross-Entropy (Log Loss)** cost function:

$$J(\theta) = -\frac{1}{m} \sum_{i=1}^m \left[y^{(i)} \log(h_\theta(x^{(i)})) + (1 - y^{(i)}) \log(1 - h_\theta(x^{(i)})) \right] \quad (6)$$

The optimization algorithm used is **Mini-Batch Gradient Descent** combined with the **Backpropagation** algorithm for efficient gradient computation.

Update Rule (Gradient Descent)

For each epoch and each batch of data, the parameters are updated iteratively:

$$\theta := \theta - \alpha \nabla_\theta J(\theta) \quad (7)$$

Where α is the optimized learning rate (e.g., $\alpha = 0.005$) and $\nabla_\theta J(\theta)$ is computed by applying the chain rule from the output layer backward.

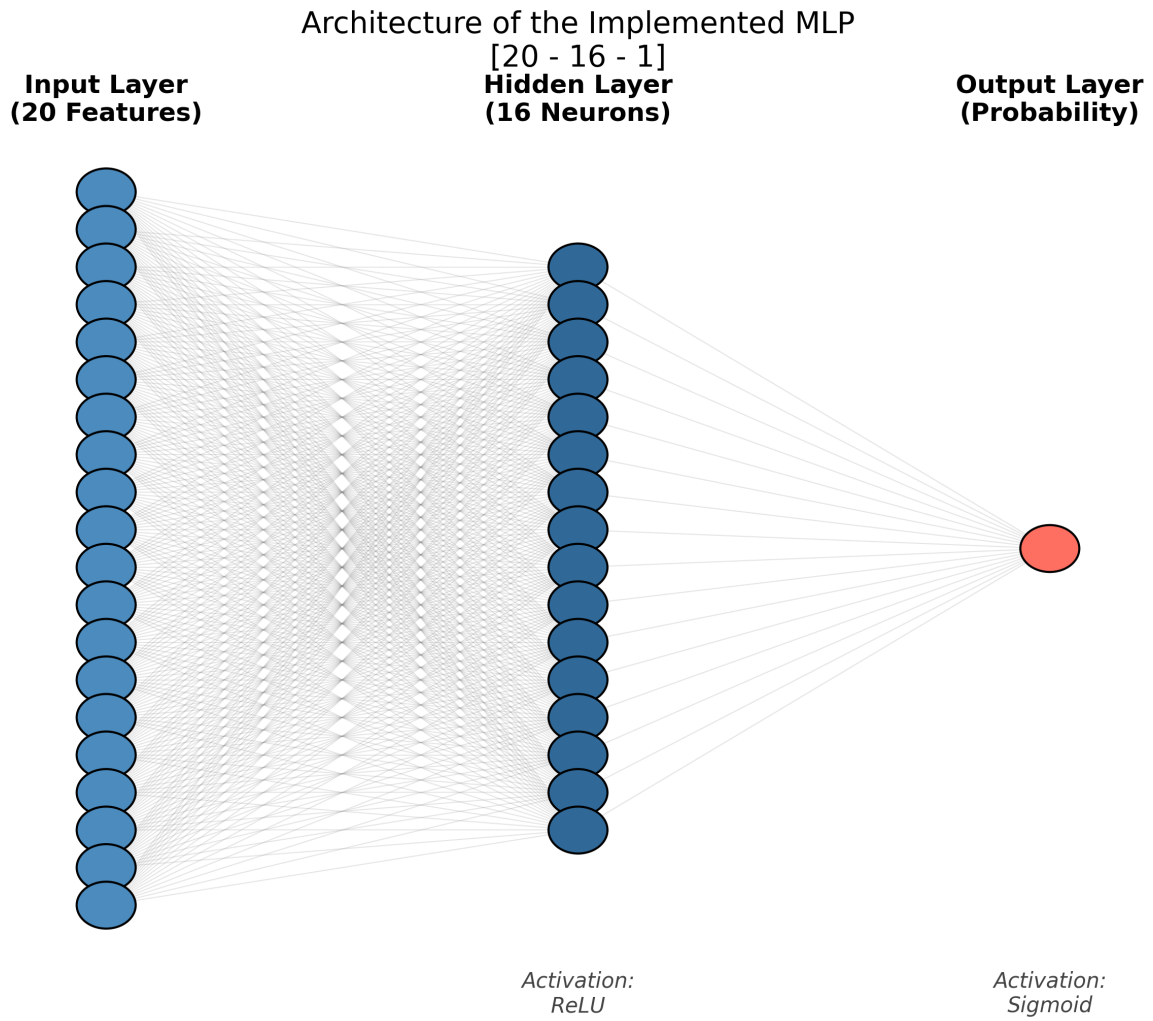


Figure 1: Diagram of the implemented Multilayer Perceptron architecture.

Acknowledgment of AI Assistance

Generative AI tools were consulted during the preparation of this document. This assistance was primarily utilized for refining the language and expression of technical definitions, as well as for assistance with Python syntax and the generation of code for data visualization. The author assumes full responsibility for the final implementation, validation, and interpretation of all code and the resulting analysis presented herein.